LANGGRAPH

Iterative Workflows

Video 8 -- Comprehensive Visual Notes

Detail	Value
Topic	Iterative (Looping) Workflows -- Self-Correcting Agents
Core Idea	Cyclic edges: downstream node connects BACK to upstream node
Key Pattern	Generator -> Evaluator -> Optimizer -> (loop back)
Case Study	Automated Tweet Generator with ruthless critic
Safety	max_iterations counter to prevent infinite loops
Prerequisites	V7 (Conditional) + Reducers from V6

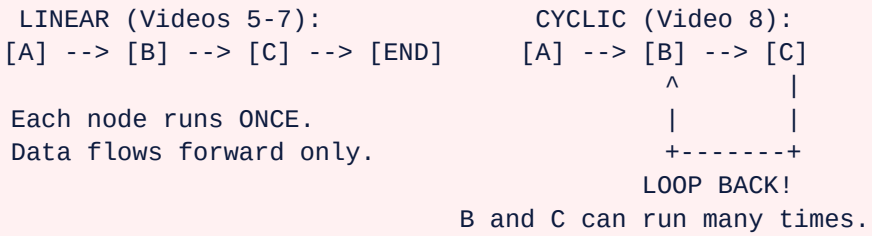
>> What You Will Learn

How cyclic edges create loops in LangGraph. The Generator-Evaluator-Optimizer pattern for self-correcting output. The critical max_iterations safety valve. Reducer-based history tracking for debugging iteration evolution. Extra: separation of LLM personas, cost analysis, debugging tips, and full code templates.

1. Core Concept: What is an Iterative Workflow?

An iterative workflow allows an agent to **repeat tasks** until a quality threshold is met. This is essential for self-correcting systems -- "Keep rewriting this email until it is professional." In programming terms, this is a `while` loop inside a graph.

Linear vs. Cyclic (Looping) Graphs



Analogy: Linear = writing an essay once and submitting. Iterative = writing a draft, having a teacher review it, revising based on feedback, resubmitting, and repeating until the teacher says "approved."

Feature	Linear Graphs (V5-V7)	Iterative Graphs (V8)
Flow direction	Forward only	Forward + backward (loops)
Node execution	Each node runs once	Loop nodes run multiple times
Quality model	Zero-shot (first attempt)	Iterative refinement
Risk	None	Infinite loops if no exit
Edge type	Static / Conditional	Backward edge + conditional exit
Programming analog	if/else, function chain	while loop

!! Two Requirements for Every Loop

1. A BACKWARD EDGE: an edge from a downstream node back to an upstream node (e.g., Optimizer -> Evaluator). This creates the cycle. 2. An EXIT CONDITION: a conditional edge that checks whether to continue looping or break out to END. Without this, the loop runs forever.

2. The Generator-Evaluator-Optimizer Pattern

The core architecture for iterative refinement uses three distinct roles in a feedback loop.

1

Generator (Creates First Draft)

An LLM produces initial content from a topic. This runs ONCE at the start. Subsequent rewrites are handled by the Optimizer, not the Generator.

2

Evaluator (The "Ruthless Critic")

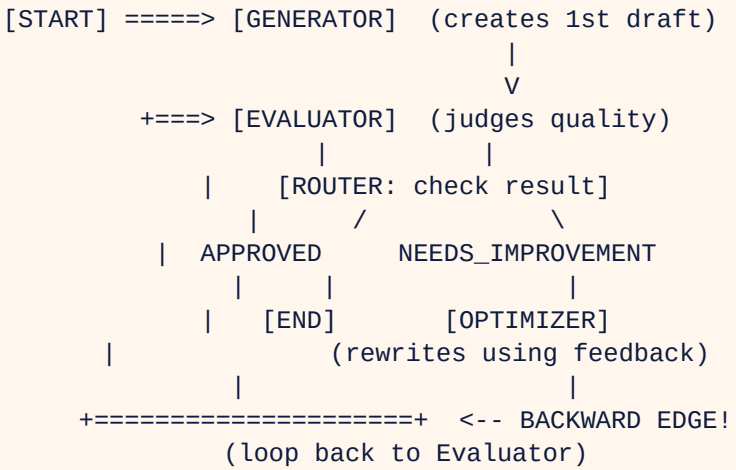
A SECOND LLM judges the content against strict criteria. Returns APPROVED or NEEDS_IMPROVEMENT + detailed feedback. This node runs EVERY iteration of the loop.

3

Optimizer (The Rewriter)

A THIRD LLM takes the current content + evaluator feedback and rewrites to fix issues. The improved version is sent BACK to the Evaluator for re-judgment, creating the loop.

The Complete Loop Architecture



! Extra: Separation of Concerns -- Why 3 LLM Personas?

Using the SAME LLM for generation AND evaluation creates bias -- it tends to approve its own work. Distinct personas yield genuine critique. The Generator is creative, the Evaluator is strict, the Optimizer is surgical. This mirrors author -> editor -> reviser in publishing. You can use different models (e.g., GPT-4o for evaluation, GPT-4o-mini for generation) or different system prompts for the same model.

3. Case Study: Automated Tweet Generator

The lecture builds a system to generate funny, original tweets using a quality feedback loop.

Tweet Generator -- Iteration Walkthrough

```
Topic: "AI in India"
ITERATION 1:
Generator:  "AI is changing India! #AI #Tech"
Evaluator:  NEEDS_IMPROVEMENT
Feedback:   "Has hashtags. Not funny. Generic."
Optimizer:  "India taught AI yoga, now Siri
             does downward-facing data"
ITERATION 2:
Evaluator:  NEEDS_IMPROVEMENT
Feedback:   "Funnier! But needs more originality."
Optimizer:  "India didn't adopt AI -- AI adopted
             India after tasting the biryani"
ITERATION 3:
Evaluator:  APPROVED!
--> [END]  Tweet published!
```

Evaluator Criteria

Criterion	What Evaluator Checks	Fail Example
Humor	Is it genuinely funny?	"AI is cool" -- not funny
Originality	Not cliche or overused?	"AI is the future" -- cliche
No Hashtags	Clean text, no #tags	"Great #AI #Tech" -- has tags
Length	Under 280 characters	300+ char essay -- too long
Tone	Matches casual Twitter voice	Too formal for social media

State Definition for the Tweet Loop

```
from typing import Annotated, TypedDict
import operator

class TweetState(TypedDict):
    topic: str              # Input topic
    tweet: str              # Current version (overwritten)
    feedback: str           # Latest evaluator feedback
    decision: str           # APPROVED or NEEDS_IMPROVEMENT
    iteration: int          # Safety counter

    # HISTORY via Reducers (from Video 6!)
    tweet_history: Annotated[list[str], operator.add]
    feedback_history: Annotated[list[str], operator.add]
```

4. Node Implementations

Generator Node (Runs Once at Start)

```
def generate_tweet(state: TweetState) -> dict:
    prompt = f"Write a funny, original tweet about: {state['topic']}"
    result = llm.invoke(prompt)
    return {
        "tweet": result.content,
        "tweet_history": [result.content],   # Append to history
        "iteration": 1,
    }
```

Evaluator Node (Runs Every Iteration)

```
from pydantic import BaseModel, Field

class EvalResult(BaseModel):
    decision: str = Field(description="APPROVED or NEEDS_IMPROVEMENT")
    feedback: str = Field(description="Specific issues to fix")

eval_llm = llm.with_structured_output(EvalResult)

def evaluate_tweet(state: TweetState) -> dict:
    prompt = (
        f"You are a RUTHLESS tweet critic.\n"
        f"Evaluate: {state['tweet']}\n"
        f"Criteria: Humor, Originality, No Hashtags."
    )
    result = eval_llm.invoke(prompt)
    return {
        "decision": result.decision,
        "feedback": result.feedback,
        "feedback_history": [result.feedback],
    }
```

Optimizer Node (Rewrites Using Feedback)

```
def optimize_tweet(state: TweetState) -> dict:
    prompt = (
        f"Rewrite this tweet to fix the issues:\n"
        f"Tweet: {state['tweet']}\n"
        f"Feedback: {state['feedback']}\n"
        f"Keep it funny, original, no hashtags."
    )
    result = llm.invoke(prompt)
    return {
        "tweet": result.content,
        "tweet_history": [result.content],
        "iteration": state["iteration"] + 1,  # INCREMENT!
    }
```

! Generator vs. Optimizer -- Key Difference

Generator: Creates from scratch (takes topic, produces first draft). Runs ONCE. Optimizer: Rewrites existing content (takes tweet + feedback, produces improved version). Runs in EVERY loop iteration. They have DIFFERENT prompts and roles. The Optimizer also increments the iteration counter -- this is critical for the safety valve.

5. Routing Logic and Graph Wiring

The Router: Loop Exit Decision

```
MAX_ITERATIONS = 5

def should_continue(state: TweetState) -> str:
    """Exit if approved OR max iterations reached."""
    if state["decision"] == "APPROVED":
        return "done"         # Quality met!
    if state["iteration"] >= MAX_ITERATIONS:
        return "done"         # Force exit (safety valve)
    return "loop"             # Continue improving
```

Complete Graph Wiring

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(TweetState)

# Add nodes
graph.add_node("generator", generate_tweet)
graph.add_node("evaluator", evaluate_tweet)
graph.add_node("optimizer", optimize_tweet)

# Forward edges
graph.add_edge(START, "generator")
graph.add_edge("generator", "evaluator")

# CONDITIONAL EDGE: the loop decision point
graph.add_conditional_edges(
    "evaluator",           # Source: after evaluation
    should_continue,       # Router function
    {"done": END,          # Exit the loop
     "loop": "optimizer"}  # Continue loop
)

# THE BACKWARD EDGE: creates the cycle!
graph.add_edge("optimizer", "evaluator")
#   downstream ^^^^^^^^^^   ^^^^^^^^^^ upstream

app = graph.compile()
```

Edge Map: What Each Edge Does

```
  START =======> generator      (static, runs once)
 generator ====> evaluator     (static, first check)
evaluator ====> ???           (CONDITIONAL: done/loop)
       if done  ==> END           (exit the loop)
     if loop  ==> optimizer     (continue refining)
   optimizer ====> evaluator     (BACKWARD EDGE!)
   This last line is what makes the graph CYCLIC.
        Without it: optimizer is a dead end.
 With it: optimizer -> evaluator -> optimizer -> ...
```

! How LangGraph Handles Cycles

LangGraph's Pregel execution engine handles cycles natively. Each time the Evaluator runs, it is a new 'Superstep.' The state is updated between iterations. LangGraph does NOT re-run from START -- only the loop nodes (Evaluator + Optimizer) repeat.

6. Critical: Preventing Infinite Loops

The **most dangerous risk**: the evaluator might NEVER approve the content. The loop runs forever, burning API credits.

The Infinite Loop Problem

```
          WITHOUT max_iterations:
 Generator -> Evaluator: NEEDS_IMPROVEMENT
-> Optimizer -> Evaluator: NEEDS_IMPROVEMENT
-> Optimizer -> Evaluator: NEEDS_IMPROVEMENT
-> Optimizer -> Evaluator: NEEDS_IMPROVEMENT
          ... FOREVER ... ($$$ burning $$$)
          WITH max_iterations = 5:
... -> Evaluator: NEEDS_IMPROVEMENT (iter 5)
 -> Router: iteration >= MAX --> FORCE EXIT
    -> [END] with best available version
```

Router Decision Flowchart

```
   [Evaluator finishes]
            |
  [should_continue router]
            |
   decision == "APPROVED" ?
  YES --> [END]  (quality met!)
            NO  -->
   iteration >= MAX_ITERATIONS ?
YES --> [END]  (forced exit, best effort)
NO  --> [Optimizer] --> back to Evaluator
```

Scenario	Iterations	Exit Reason	Quality
Nails it first try	1	APPROVED	Excellent
Needs minor fix	2-3	APPROVED	Good
Hard criteria	5 (max)	FORCE EXIT	Best effort
No max_iterations	Infinity	NEVER EXITS	$$$$ burned

>> Extra: Choosing MAX_ITERATIONS

Too low (1-2): Content barely improves beyond first draft. Too high (20+): Expensive with diminishing returns after ~5 iterations. Sweet spot: 3-7 for most LLM refinement tasks. Pro tip: Log quality scores per iteration. If quality plateaus at N, set max = N+1.

7. State History with Reducers

Without history tracking, you only see the final tweet. Reducers (from Video 6) let you preserve every version for debugging and display.

Without vs. With History Tracking

```
WITHOUT history (tweet overwritten):
   Iter 1: tweet = "AI is cool #AI"
Iter 2: tweet = "India taught AI yoga"   (v1 LOST!)
Iter 3: tweet = "AI adopted India"       (v2 LOST!)
   --> You only see the final version!
    WITH history (reducer appends):
            tweet_history = [
 "AI is cool #AI",           # v1 preserved
 "India taught AI yoga",     # v2 preserved
  "AI adopted India",         # v3 (final)
                        ]
--> Full evolution visible for debugging!
```

```python
# Dual State Strategy: current value + history list

class TweetState(TypedDict):
    tweet: str     # Overwritten each iteration (for nodes)
    tweet_history: Annotated[list[str], operator.add]  # Appended
    feedback_history: Annotated[list[str], operator.add]

# Every node returns BOTH current and history:
def optimize_tweet(state) -> dict:
    new_tweet = llm.invoke(...).content
    return {
        "tweet": new_tweet,                # Overwrites
        "tweet_history": [new_tweet],       # Appends via reducer
        "iteration": state["iteration"] + 1,
    }

# Return [new_tweet] not new_tweet! (list for operator.add)
```

>> Extra: Debug Power of History Lists

With tweet_history and feedback_history you can: (1) See how content improved over iterations. (2) Identify which feedback led to the biggest improvement. (3) Detect if the optimizer is going in circles (similar versions). (4) Show users a 'revision timeline.' Remember: return [value] (in a list) for reducer append, just like in Video 6!

8. Extra: Complete Code Template

A **complete, runnable** iterative tweet generator with safety and history.

```
# === ITERATIVE TWEET GENERATOR (Part 1: Setup) ===
from typing import Annotated, TypedDict
import operator
from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

MAX = 5
class EvalOut(BaseModel):
    decision: str = Field(description="APPROVED or NEEDS_IMPROVEMENT")
    feedback: str = Field(description="What to fix")

class St(TypedDict):
    topic: str; tweet: str; feedback: str; decision: str
    iteration: int
    tweet_history: Annotated[list[str], operator.add]
    feedback_history: Annotated[list[str], operator.add]

llm = ChatOpenAI(model='gpt-4o-mini')
e_llm = llm.with_structured_output(EvalOut)
```

```
# === Part 2: Nodes ===
def gen(s):
    r=llm.invoke(f"Funny tweet about: {s['topic']}")
    return {"tweet":r.content,"tweet_history":[r.content],"iteration":1}

def evl(s):
    r=e_llm.invoke(f"Ruthless critic. Evaluate: {s['tweet']}")
    return {"decision":r.decision,"feedback":r.feedback,
            "feedback_history":[r.feedback]}

def opt(s):
    r=llm.invoke(f"Rewrite: {s['tweet']}\nFix: {s['feedback']}")
    return {"tweet":r.content,"tweet_history":[r.content],
            "iteration":s["iteration"]+1}
```

```
# === Part 3: Router + Graph ===
def route(s):
    if s["decision"]=="APPROVED" or s["iteration"]>=MAX:
        return "done"
    return "loop"

g=StateGraph(St)
g.add_node("gen",gen); g.add_node("evl",evl); g.add_node("opt",opt)
g.add_edge(START,"gen"); g.add_edge("gen","evl")
g.add_conditional_edges("evl",route,{"done":END,"loop":"opt"})
g.add_edge("opt","evl")  # <-- THE BACKWARD EDGE (loop!)

app=g.compile()
r=app.invoke({"topic":"AI in India","tweet":"","feedback":"",
  "decision":"","iteration":0,"tweet_history":[],"feedback_history":[]})
print(f"Final ({r['iteration']} iters): {r['tweet']}")
for i,t in enumerate(r["tweet_history"],1):
    print(f"  v{i}: {t}")
```

9. Extra: Common Pitfalls

Pitfall	What Goes Wrong	Fix
No max_iterations	Infinite loop, $$$ burned	Add counter + force exit in router
No conditional exit on loop	Graph cycles forever	Use add_conditional_edges on evaluator
Same LLM for all roles	Evaluator approves own work (bias)	Use different prompts or models
Not incrementing counter	Counter stays 0, max check fails	Optimizer must increment iteration
No history tracking	Can't debug why loop ran N times	Use Annotated[list, operator.add]
Criteria too strict	Never approves, always hits max	Calibrate criteria or relax after N loops
Criteria too lenient	Approves bad content on iter 1	Make criteria more specific
Forgetting initial state	KeyError on first run	Pass ALL keys with defaults in invoke()

>> Extra: Cost Analysis for Loops

Each iteration: 1 Evaluator call + 1 Optimizer call = 2 LLM calls. With MAX=5: worst case = 1 (gen) + 5 x 2 (loop) = 11 calls. GPT-4o-mini: ~$0.001-0.005 per full run. GPT-4o: ~$0.10-0.50 per run. 1000 users/day with GPT-4o-mini: ~$1-5/day (cheap!). Always estimate worst-case cost before deploying iterative agents.

10. Where Iterative Workflows Fit

Pattern	Video	Loops?	Key Edge	Complexity
Sequential	V5	No	Static	* Easy
Parallel	V6	No	Fan-out/in	** Medium
Conditional	V7	No	Conditional	** Medium
Iterative (THIS)	V8	YES	Backward + conditional	*** Hard
Full Agent	Later	Yes	All combined	**** Expert

>> Your Complete Edge Toolkit

V5: Static edge (A -> B always). V6: Fan-out/fan-in (A -> B AND C in parallel). V7: Conditional edge (A -> B OR C, router decides). V8: Backward edge + conditional exit (B -> A loop). You now have ALL four fundamental edge types! Every LangGraph agent you will ever build is a combination of these four building blocks.

11. Quick Revision Checklist

Tag	Question
Concept	What is an iterative workflow? How does it differ from linear?
Cyclic Edge	What single line of code creates a loop?
3 Roles	Name the 3 nodes in Generator-Evaluator-Optimizer.
Exit	What are the TWO exit conditions for the loop router?
Safety	Why is max_iterations essential? What happens without it?
Router	Write the should_continue function from memory.
History	How do you track tweet_history using reducers?
Personas	Why use different LLM personas for generator vs critic?
Cost	How many API calls does worst-case 5-iteration loop make?
Combine	Can you combine loops with parallel and conditional edges?

Cheat Sheet: Iterative Workflow Patterns

What	Code
Backward edge	graph.add_edge('optimizer', 'evaluator')
Loop exit router	if decision=="APPROVED" or iter>=MAX: return "done"
History via reducer	tweet_history: Annotated[list[str], operator.add]
Append to history	return {"tweet_history": [new_tweet]} # list!
Increment counter	return {"iteration": state["iteration"] + 1}
Pydantic for eval	eval_llm = llm.with_structured_output(EvalModel)
Initialize state	invoke({...,"iteration":0,"tweet_history":[]})

What's Next?

You now have ALL four fundamental edge types: static, parallel, conditional, and cyclic. Upcoming videos combine these into full autonomous agents with tool use, memory, and human-in-the-loop control. Make sure you can draw the Generator-Evaluator-Optimizer loop and explain the two exit conditions from memory!